

PAUT Explainable-AI Digital Twin

Architectural Solution Approach

Weld defect inspection from Phased Array Ultrasonic Testing (TFM) images — detection, classification, characterization, explainable AI, and a Digital-Twin health dashboard.

A complete, end-to-end description of the system we built: what each part does, how it is connected, and **why** each design choice was made. This document describes the system exactly as implemented in the repository (it is not a wish-list — every block below maps to real code).

1. Problem statement

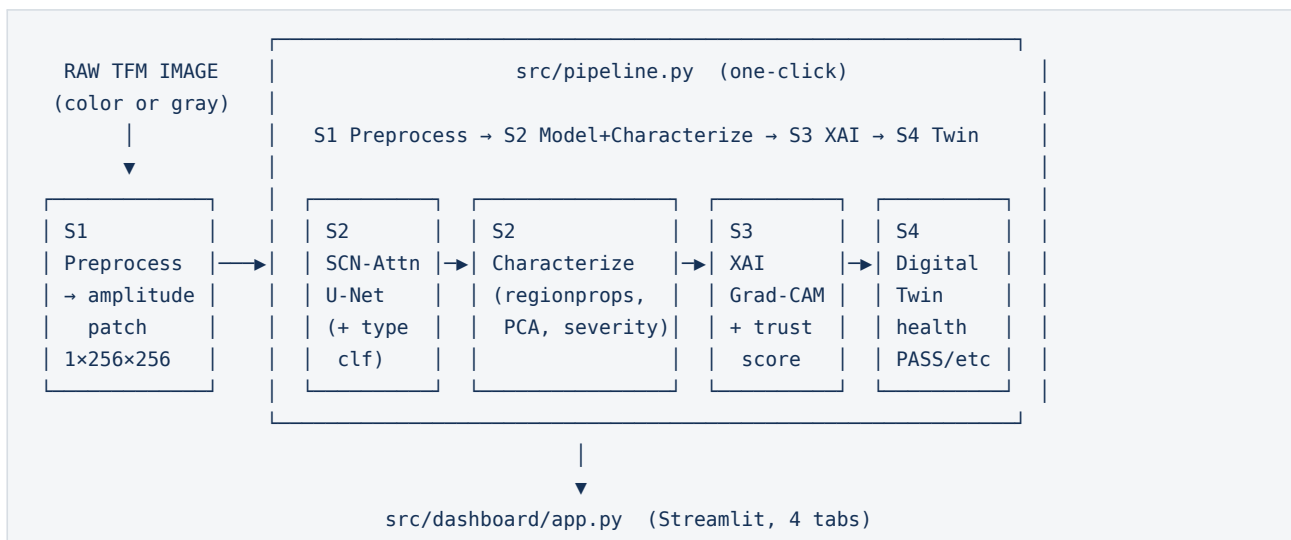
Welds inspected with **Phased Array Ultrasonic Testing (PAUT)** are imaged as **TFM (Total Focusing Method)** amplitude maps. A human inspector must look at each image, decide whether a defect is present, say **what kind** it is (here: **porosity** vs **slag**), measure it, and judge whether the weld passes. This is slow, subjective, and hard to audit.

Our goal: an **Explainable-AI Digital Twin** that does all four steps automatically and shows its reasoning —

1. **Detect & classify** the defect,
2. **Characterize** it (size, shape, orientation, severity) from its segmentation mask,
3. **Explain** the prediction with model-appropriate XAI (so an engineer can trust it), and
4. **Roll it up** into a Digital-Twin health view (PASS / REVIEW / FAIL) on a dashboard.

Key constraint that shaped the whole design: very limited, noisy data (1050 images) and a CPU-only development laptop. This is why we chose a **scattering prior** (generalizes from small data) and a **develop-local / train-in-cloud** workflow.

2. High-level architecture (the big picture)



The system is split into **six milestones**, each an independent module with its own config, tests, and CLI. They are chained by the one-click `pipeline.py`, and the result is presented by the Streamlit dashboard. An independent **scattering-feature type classifier** sits alongside the model to decide defect **type** (the neural head was too weak), and a **data-ablation harness** proves the value of the scattering prior.

3. The data layer (Milestone 1 + 2)

3.1 Dataset

- data/raw/porosity/ — 525 porosity images (G...-named)
- data/raw/slag/ — 525 slag images (GS...-named)
- **Total: 1050 images, balanced 525 / 525**, grayscale, small/faint defects on a near-black field.

Data-integrity story (important). *The porosity folder originally held 256 GS...-named files that were *byte-identical duplicates of slag images*. This made the two classes contradictory and*

pinned type-classification at 50% (chance). We built scripts/check_contamination.py (MD5 cross-folder dedup) to find them, quarantined them, and added genuine porosity images to restore a

*clean 525 / 525. This is a real example of debugging the *data*, not just the model.*

3.2 Preprocessing — src/data/preprocess.py (config: configs/preprocess.yaml)

Every raw image becomes **one normalized single-channel 256×256 float32 amplitude patch**:

1. **Source detection** — grayscale vs color (detect_source_type).
2. **Color → amplitude** — a TFM image is a scalar amplitude field; color is only a colormap.

Grayscale takes the luminance path; a colormap-inverse path is wired in for future color data with no code change (to_amplitude).

3. **Two-stage border crop** (crop_border) — raw frames have a white matplotlib border around the

dark data region. Stage 1 detects and removes the white frame (content-based, per image); stage 2 trims a residual bright anti-aliased rim band (_trim_bright_band). A localized bright defect touching the edge is **kept** (verified by a unit test).

4. **Per-image min-max normalize** to [0,1] — cancels gain/brightness differences and lifts faint

defects (normalize).

5. **Resize** to 256×256 (resize_patch).

3.3 Leakage-safe split — src/data/split.py

- **70 / 15 / 15** train/val/test, **stratified by class, fixed seed** (deterministic).
- **Weld-group-aware**: filenames like G100_1 / _2 / _3 are different *views of the same weld*.

All views of a weld stay in **one** split, so near-duplicate views never leak across train/val/test (weld_id_from_stem strips the trailing _<digit>). A built-in guard asserts no weld spans two splits.

- **Split happens before any augmentation** — augmentation is applied to the **train split only**.

Output of Milestone 1: data/processed/<class>/<id>.npz patches + manifest.csv (the contract handed to every downstream module).

3.4 Stage-1 pseudo-labels — src/data/pseudo_label.py (config: configs/pseudo_label.yaml)

We have no hand-drawn masks, so we bootstrap **approximate** masks from the amplitude: **Otsu threshold (with a floor) → OpenCV morphology (close/open) → fill → drop tiny components.**

- **Otsu**, because the histogram is ~99% background, so the threshold lands in the gap and isolates

the bright defect core.

- **OpenCV morphology, not scipy** — `scipy.ndimage.binary_closing` over-erodes thin slag (~3×

shrink); `cv2.MORPH_CLOSE` closes correctly (`_drop_small_components` avoids a deprecated `skimage` API).

- **Every image gets a mask** — each image contains a defect, so if the standard pipeline finds

nothing we retry keeping single pixels (`fallback_keep_singletons`). Result: 0 empty masks.

Output of Milestone 2: data/processed/masks/<class>/<id>.npz label maps (0=background, 1=porosity, 2=slag) + overlays to eyeball quality.

3.5 Dataset & augmentation — src/data/dataset.py

PAUTDataset loads patch + mask + class label, with a `fraction` parameter (for the ablation). Augmentor (train split only) applies **physics-aware augmentation**: flips, small rotations, gain changes, speckle noise, and depth-attenuation — transformations that reflect real ultrasonic variability rather than generic image jitter.

4. The core model (Milestone 3)

4.1 SCN-Attention U-Net — src/models/scn_attention_unet.py

The heart of the system: a **fixed Wavelet-Scattering branch fused into an Attention U-Net**, with three heads. (`build_model(cfg)` constructs it from `configs/model.yaml`.)

```
input 1x256x256 amplitude patch
|
├ (a) Attention U-Net encoder — E1(32) → E2(64) → E3(128) → E4(256) → bottleneck(512)
|
└ (b) fixed Wavelet Scattering — S (multi-channel tensor) + global vector
    |
    per-scale fusion (SCNFusion): concat(E_i, resize(1x1Conv(S))) → 1x1Conv → GroupNorm → CBAM
    |
    attention-gated decoder ← up4 → up3 → up2 → up1
    |
    └ Head 1 SEGMENTATION 1x1 conv on last decoder layer → 3x256x256 (bg / porosity / slag)
    └ Head 2 CLASSIFICATION [bottleneck-pool ⊗ global-SCN-vec] → MLP → 2 logits (porosity/slag)
    └ Head 3 SIZE REGRESSION (optional, OFF – size is measured from the mask instead)
```

4.2 The scattering branch — src/models/scattering.py (the central idea)

A **Wavelet Scattering Transform** (via **Kymatio**, $J=2$, $L=8$, $\text{order}=2$) with **frozen filters (no gradients)**. It produces a translation-invariant, deformation-stable, multi-scale texture descriptor.

- **Why scattering?** Fixed wavelet filters **cannot overfit** tiny data — that is the entire reason

for this architecture. It injects a strong, hand-designed prior so the learnable part has far less to learn. This directly matches the reference paper's small-sample strategy.

- **Only the 1×1 projection convs in the fusion are learnable**; the scattering filters never train

(locked decision §9).

- **Kymatio import workaround:** we import `ScatteringTorch2D` from

`kymatio.scattering2d.frontend.torch_frontend` directly, because `kymatio.torch` eagerly imports a 3D module needing `scipy.special.sph_harm`, which was removed in `scipy ≥ 1.16`.

4.3 Attention blocks — src/models/blocks.py

- `ConvBlock` uses **GroupNorm, not BatchNorm** — batch sizes are tiny on limited data, where

BatchNorm statistics are noisy and unreliable.

- **CBAM** (channel + spatial attention) and `AttentionGate` on the decoder skips suppress the

busy weld background. Crucially, **both cache their attention maps** (`last_spatial_map`, `last_gate_map`) so they can be surfaced **for free** as intrinsic explanations in Milestone 5.

4.4 Loss — src/models/losses.py (the bug we had to fix)

`MultiTaskLoss = 0.6·Dice-Focal(seg) + 0.3·classification + 0.1·boundary`, **plus a foreground-focused Focal-Tversky term**.

The collapse bug. With a standard Dice that **includes** background, the segmentation head

collapsed to predicting "all background" (since ~99% of pixels are background, that scores well).

Fix: make Dice/Tversky **foreground-only** (`include_background_in_dice: false`) and add

Focal-Tversky with $\beta > \alpha$ ($\beta=0.7$, $\alpha=0.3$) which heavily penalizes **false negatives** (missing a

defect). We added a regression test, `test_loss_punishes_all_background_collapse`, so this can

never silently come back.

Inverse-frequency **class weighting** is enabled as a safeguard.

4.5 Training — src/models/train.py

Cosine LR with warmup; device auto-detected (CPU-runnable, GPU-accelerated). Saves the best checkpoint by **validation foreground-Dice**. Flags for every workflow: `--limit N` (CPU

smoke-test), --overfit N (prove the model *can* learn), --data-fraction (ablation), --no-scattering (baseline), --ckpt-dir (never overwrite the production checkpoint). Final checkpoint: checkpoints/scn_attn_unet_best.pt (selected by best validation foreground-Dice); test foreground Dice $\approx$ **0.57** (porosity 0.45, slag 0.68).

5. Defect type — the scattering-feature classifier

(src/models/type_classifier.py)

Why a separate classifier? *The U-Net ***detects*** defects well but ***types*** them poorly — its global-average-pooled classification head reaches only ~ 0.65 balanced accuracy on test, and the*

seg-derived type is no better (~ 0.65 , with porosity mislabelled as slag about half the time — it

throws away the spatial texture information that distinguishes the two). In the short data-ablation

runs the head sat even lower, near chance ($\sim 50\%$).

Porosity-vs-slag is a **whole-image texture property**, and the fixed scattering transform is exactly the right small-sample texture descriptor. So:

1. Extract the **global scattering vector** (mean + std of scattering coefficients over space) for

each image (scatter_features).

2. **Log-scattering renormalise** (log of the coefficients — the standard Mallat/Bruna step),

standardise, then fit a **tiny one-hidden-layer MLP** — i.e. the "scattering + small neural classifier" of the IWSCN reference paper. (A plain logistic regression gives ~ 0.80 ; the log-renormalisation + small MLP lifts it to ~ 0.88 .)

3. Result: **test balanced accuracy $\approx$ 0.88** (porosity 0.86, slag 0.90) — versus ~ 0.65 for the

neural head. Selected by 5-fold CV on the train split (CV 0.87 $\approx$ test 0.88, so it generalises, not overfit). The scattering filters stay fixed (J=2) — **only the small classifier changed, the U-Net architecture is untouched**. Trains on **CPU in minutes** (no GPU, no checkpoint needed).

Division of labour: the **U-Net does detection** (where the defect is), the **scattering classifier does type** (porosity vs slag). The pipeline relabels all detected foreground pixels to the image-level type (valid because every image is single-type, design decision §2).

6. Characterization (Milestone 4) — src/characterize/characterize.py

Turns each defect **mask** into engineering measurements (one record per connected component):

- **Size & shape** via skimage.regionprops: area, equivalent diameter, axis_major_length /

`axis_minor_length`, aspect ratio, eccentricity.

- **Orientation** via **PCA** on the blob's pixels.
- **Location**: centroid in weld coordinates.
- **Severity**: a tunable score (`configs/characterize.yaml`).

Design notes:

- `pixel_to_mm` **defaults to 1.0** → measurements reported in **pixels**; a real mm scale plugs in

later with no code change. Scale-free fields (orientation, aspect ratio, eccentricity) are correct regardless.

- **Severity thresholds are placeholders, NOT acceptance criteria** — they must be calibrated to a

real standard (ISO 5817 / ASME) with a domain expert before any pass/fail use. This is stated everywhere it appears.

Output: per-image `characterization/<id>.json` (the S2 → S4 contract) + a summary CSV + annotated overlays.

7. Explainable AI (Milestone 5) — `src/xai/`

Each explanation is **matched to the head it explains**:

- **Seg-Grad-CAM / Grad-CAM++** (`gradcam.py`) on the **last decoder conv** for the segmentation head

→ **where** in the image drove the detected defect.

- **Grad-CAM** on the **bottleneck** for the classification head.
- **Intrinsic attention** (`attention.py`) — the model's own **CBAM** spatial maps and

Attention-Gate coefficients, surfaced for free (faithful by construction).

- **Faithfulness metrics** (`faithfulness.py`) — **deletion** AUC (lower=better), **insertion** AUC

(higher=better), and **pointing-game** → combined into a single **trust score** per explanation. For segmentation we use **region-based** scoring so a tiny defect isn't swamped by background.

Result on a 16-image validation sample: mean trust ≈ 0.91 (deletion 0.12, insertion 0.86, pointing-game 1.00) — the Grad-CAM explanations are genuinely faithful, not decorative.

8. Digital Twin + dashboard (Milestone 6)

8.1 The twin — `src/twin/twin.py`

Rolls per-defect severities into a single weld view:

```
health_index = 1 - min(1, Σ severity_score / capacity)    (1 = pristine)

status = FAIL    if any critical defect
```

```
= REVIEW if health < review_health OR ≥ N moderate defects
= PASS otherwise
```

Driven by **mask-derived defect presence + severity**, never by a "normal" class. Thresholds are placeholders pending standard calibration.

8.2 One-click pipeline — `src/pipeline.py`

The `Pipeline` class loads the model + type classifier **once**, then `analyze(image)` runs the full chain (`S1 → S2 → S3 → S4`) and returns one structured dict with everything the dashboard needs. `--out defects.json` dumps a JSON-safe version (big arrays stripped).

8.3 Dashboard — `src/dashboard/app.py` (**Streamlit, 4 tabs**)

- **Overview** — colored PASS/REVIEW/FAIL status banner + weld map with severity-colored defect

markers.

- **Explainability** — original image, Seg-Grad-CAM heatmap, intrinsic attention map, trust score.

- **Defects** — per-defect table + cards (size / orientation / severity / confidence) + `download-defects.json` button.

- **Details** — a **model-accuracy panel** (read from `data/processed/model_metrics.json`) shown

separately from this weld's health index, plus a sidebar pixel→mm scale input. (This separation fixed a real confusion: weld *health* is this part's condition; model *accuracy* is how good the model is — two different numbers.)

9. Data-ablation experiment — `scripts/run_ablation.py`

To prove the scattering prior earns its place, we trained **SCN vs a plain Attention-U-Net baseline** (`--no-scattering`) at **100 / 50 / 25 / 10 %** of the training data (8 runs, 60 epochs, Colab T4). It writes to `checkpoints/ablation/` so the production checkpoint is never touched, and produces `ablation_results.csv` + `ablation_curve.png`.

Data used	SCN val Dice	Plain U-Net val Dice
10% (74 imgs)	0.541	0.514
25% (184 imgs)	0.540	0.567
50% (368 imgs)	0.580	0.664
100% (735 imgs)	0.642	0.689

Finding: at the **lowest data fraction (10%)** the scattering prior wins (0.541 vs 0.514), confirming it regularizes in the low-data regime that is realistic for PAUT. With more data the plain U-Net overtakes it (the fusion complexity is no longer worth it). In these short ablation runs both classification heads stayed near chance (~50% val accuracy); even the fully-trained production head only reaches ~0.65 balanced — which is *exactly* why type is decided by the standalone scattering classifier (0.88) instead.

The same curriculum applied to the **type classifier** (log-scattering + tiny MLP) shows it degrades **gracefully** and stays well above chance as data shrinks — **0.88 → 0.83 → 0.79 → 0.72** at 100 / 50 / 25 / 10 % of the train split (still 0.72 on just 74 images). This is the small-data robustness the fixed scattering prior is chosen for.

(scripts/ablation_type_classifier.py; curve at data/processed/ablation/type_clf_ablation.png.)

10. Engineering conventions (what makes it robust)

- **Develop local (CPU), train in cloud (GPU).** Device is always

`torch.device("cuda" if available else "cpu")` — never hard-coded. `--limit/--fraction` flags give fast CPU smoke-tests; `notebooks/train_colab.ipynb` runs full GPU training.

- **Everything config-driven** (`configs/*.yaml`) — the same code runs locally and in the cloud by

swapping a YAML.

- **Module-by-module with tests.** 6 test files (`tests/`), ~37 tests, including the loss-collapse

regression guard — each milestone is verified before the next is built.

- **Reproducibility:** fixed seeds, deterministic splits, manifests as contracts between stages.

11. End-to-end data flow (one sentence per hop)

1. **Raw image** → preprocess → **1×256×256 amplitude patch** (+ manifest row).
2. Patch → **SCN-Attention U-Net** → **segmentation mask** (detection) + features.
3. Patch → **scattering classifier** → **defect type** (porosity / slag), applied to the mask.
4. Mask → **characterization** → per-defect **size / shape / orientation / severity** records.
5. Patch + mask → **XAI** → **Grad-CAM heatmap + trust score**.
6. Defect records → **twin** → **health index + PASS/REVIEW/FAIL**.
7. Everything → **Streamlit dashboard** (4 tabs).

12. Honest limitations (and the fix for each)

Limitation	Cause	Fix / status
Porosity segmentation Dice ~0.45 (test)	Pseudo-labels reduce porosity to tiny bright cores	Hand-label ~30–50 val images (biggest lever)
Neural classification head ~0.65 balanced	Global-average pooling discards shape	Routed around via scattering classifier (0.88)
Severity / pass-fail thresholds	No calibrated standard yet	Tunable placeholders; calibrate to ISO 5817 / ASME
<code>pixel_to_mm</code> unknown	Scale not provided	Defaults to 1.0 (pixels); a real value plugs in, no code change

Each is documented in code and `design_decisions.md`, so nothing is hidden.

Appendix A — Data-Ablation Curve

SCN-Attention U-Net (solid) vs plain Attention U-Net baseline (dashed), validation Dice and classification accuracy against training-data size.

